

1) Implementation of an environmental monitoring system based on a Microcontroller, Sensors and a TFT display

Group No. B2**

Date 15/05/25

Last Name/First Name	ID Number
Miotto Isabella	2066715
Orso Marco	2074059
Sette Andrea	2066747

Purpose of the laboratory: to build a simple environmental monitoring station based on a microcontroller board, TFT display, analog humidity sensor and digital (I2C) temperature and light sensors.

Instrumentation needed:

- Microcontroller board
- Breadboard
- Adafruit HX8357 Display

Components needed:

Component type	Manufacturer Code/Value
Analog temperature sensor	TMP36, Analog Devices
Analog humidity sensor	HIH-5030-001, Honeywell
Digital temperature sensor	DS1621, Maxim
Ambient light sensor	TSL2561, TAOS
3.5" TFT display	Adafruit HX8357, 320x480 resolution
Microcontroller board	
Breadboard and cables	

The circuit is powered through the USB port of the pc.

1.1) First Experiment

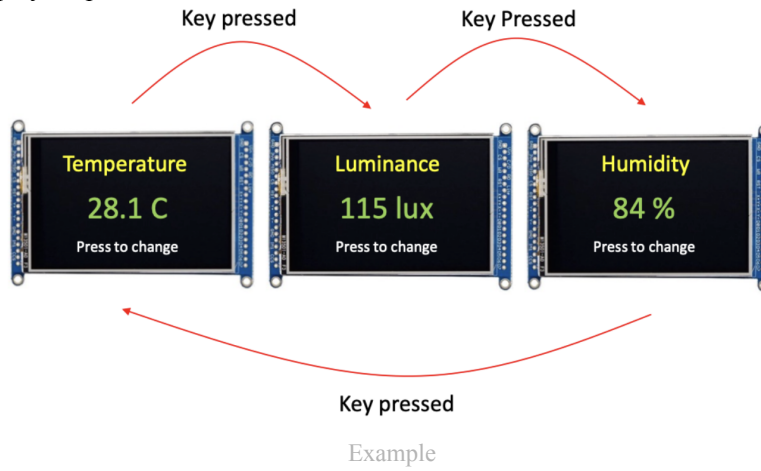
The purpose of the experiment is to build the circuit and write a program that displays the temperature, humidity and ambient light level. The parameter displayed on the screen should update only when its value changes.

We recommend to

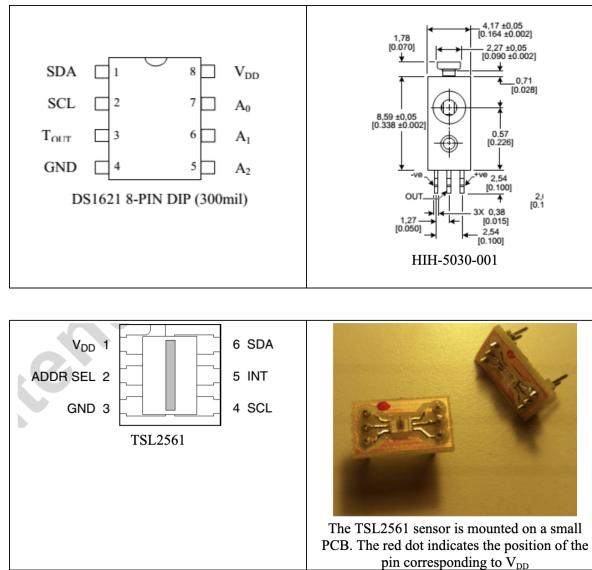
1. write the functions that take the reading from each sensor
2. verify their operation with the serial monitor
3. use such functions to build the complete program.

Pressing a key should allow the screen reading to change, as shown in the figure.

An example of the display output is shown below.



The pinout of DS1621 (Maxim), HIH-5030-001, (Honeywell) and TSL2561 (TAOS) sensors is given below:



Pinout

Report the code in the following box, appropriately annotated:

```
// DEFINIZIONE COLORI
#define HX8357_BLACK    0x0000
#define HX8357_BLUE    0x001F
#define HX8357_RED     0xF800
#define HX8357_GREEN   0x07E0
#define HX8357_CYAN    0x07FF
#define HX8357_MAGENTA 0xF81F
#define HX8357_YELLOW  0xFFE0
#define HX8357_WHITE   0xFFFF

// INCLUSIONE LIBRERIE
#include <SPI.h>
#include "Adafruit_GFX.h"
#include "Adafruit_HX8357.h"
#include <Wire.h>

// DEFINIZIONE PIN DISPLAY
#define TFT_CS    10
#define TFT_DC    9
#define TFT_RST   8

Adafruit_HX8357 tft = Adafruit_HX8357(TFT_CS, TFT_DC, TFT_RST);
```

```

// DEFINIZIONE SENSORI
#define DS1621_ADDR 0x48
#define TSL2561_ADDR 0x39

#define TSL2561_COMMAND_BIT 0x00
#define TSL2561_CONTROL_POWERON 0x03
#define TSL2561_CONTROL_POWEROFF 0x00
#define TSL2561_REGISTER_TIMING 0x81
#define TSL2561_REGISTER_ADC0_LSB 0x8C
#define TSL2561_REGISTER_ADC0_MSB 0x8D
#define TSL2561_REGISTER_ADC1_LSB 0x8E
#define TSL2561_REGISTER_ADC1_MSB 0x8F
#define TSL2561_GAIN_LOW_STDINT 0x02

// VARIABILI
int PinAnalogTemperature = A0;
int PinAnalogHumidity = A1;

enum Screen { SCREEN_TEMP, SCREEN_HUMID, SCREEN_LIGHT };
Screen currentScreen = SCREEN_TEMP;

int old_value = -1000;
#define BUTTON_PIN 2

int fontSize = 3;

// IMMAGINI
const unsigned char epd_bitmap_thermometer [] PROGMEM = {
    0xff, 0xc3, 0xff, 0xff, 0x7e, 0xff, 0xfc, 0x1f, 0x7f, 0xfc, 0xff, 0x7f, 0xfc,
    0xff, 0x7f, 0xfc,
    0x1f, 0x7f, 0xfc, 0x07, 0x7f, 0xfc, 0xc3, 0x7f, 0xfc, 0x03, 0x7f, 0xfc, 0x03,
    0x7f, 0xfc, 0x83,
    0x7f, 0xfc, 0x83, 0x7f, 0xfc, 0xc3, 0x7f, 0xfc, 0xc3, 0x7f, 0xfb, 0xc3, 0xdf,
    0xff, 0x81, 0xff,
    0xde, 0x00, 0x7b, 0x78, 0x00, 0x1c, 0x78, 0x00, 0x1c, 0x78, 0x00, 0x1c, 0xdc,
    0x00, 0x7b, 0xdf,
    0x00, 0xfb, 0xf8, 0xff, 0x1f, 0xff, 0x00, 0xff
};

const unsigned char epd_bitmap_light [] PROGMEM = {
    0xff, 0xc3, 0xff, 0xfe, 0x3c, 0x7f, 0xfb, 0xff, 0x9f, 0xfb, 0xff, 0xdf, 0xe7,
    0xff, 0xef, 0xdf,
    0xff, 0xf3, 0xdf, 0xff, 0xf3, 0x3f, 0xff, 0xfc, 0x3c, 0xff, 0x7c, 0x3c, 0xff,
    0x7c, 0xdf, 0x7c,
    0xf3, 0xdf, 0x9b, 0xf3, 0xff, 0x9b, 0xe7, 0xf7, 0x9b, 0xef, 0xfb, 0xdb, 0x9f,
    0xfe, 0x99, 0x7f,
    0xfe, 0x00, 0x7f, 0xfe, 0xff, 0x7f, 0xfe, 0x00, 0x7f, 0xfe, 0xff, 0x7f, 0xfe,
    0x00, 0x7f, 0xff,
    0x3c, 0xff, 0xff, 0xf3, 0xff, 0xff, 0xc3, 0xff
};

const unsigned char epd_bitmap_humid [] PROGMEM = {
    0xff, 0xe7, 0xff, 0xff, 0xc3, 0xff, 0xff, 0x99, 0xff, 0xff, 0x7c, 0xff, 0xff,
    0x7c, 0xff, 0xff,
    0x7c, 0xff, 0xfe, 0xfe, 0x3f, 0xfc, 0xff, 0x3f, 0xfc, 0xff, 0x3f, 0xfb, 0xff,
    0x1f, 0xfb, 0xff,
    0x9f, 0xfb, 0xff, 0x9f, 0xfb, 0xff, 0x8f, 0xe7, 0xff, 0xe7, 0xe7, 0xff, 0xe7,
    0xe7, 0xff, 0xe7,
    0xe7, 0xff, 0xe7, 0xe7, 0xff, 0xe7, 0xe7, 0xff, 0xe7, 0xf1, 0xff, 0x8f, 0xf9,
    0xff, 0x9f, 0xfc,
    0xff, 0x3f, 0xfe, 0x00, 0x7f, 0xff, 0x00, 0xff
};

const int epd_bitmap_allArray_LEN = 3;
const unsigned char* epd_bitmap_allArray[3] = {
    epd_bitmap_thermometer,
    epd_bitmap_humid,
    epd_bitmap_light
};

```

```

};

// setup functions
void setup() {
  Serial.begin(9600);
  Wire.begin();

  displaySetup();
  DS1621Setup();
  TSL2561Setup();

  pinMode(BUTTON_PIN, INPUT);
  attachInterrupt(digitalPinToInterrupt(BUTTON_PIN), changeDisplay, RISING);
}

void displaySetup() {
  pinMode(BUTTON_PIN, INPUT_PULLUP);
  tft.begin();
  tft.setRotation(1);
  tft.fillScreen(HX8357_BLACK);
  tft.setTextSize(fontSize);
  tft.setTextColor(HX8357_YELLOW);
  tft.setCursor(30, 100);
  tft.print("Initializing...");
  delay(1500);
  tft.fillScreen(HX8357_BLACK);
}

void DS1621Setup() {
  Wire.beginTransmission(DS1621_ADDR);
  Wire.write(0xAC);
  Wire.write(0x02);
  Wire.endTransmission();
  Wire.beginTransmission(DS1621_ADDR);
  Wire.write(0xEE);
  Wire.endTransmission();
}

void TSL2561Setup() {
  Wire.beginTransmission(TSL2561_ADDR);
  Wire.write(0x80); //CONTROL reg address
  Wire.write(0x03);
  Wire.endTransmission();
}

// loop functions
void loop() {
  float value;
  const char* unit;

  switch (currentScreen) {
    case SCREEN_TEMP:
      value = DigitalTemperature();
      unit = " C";
      break;

    case SCREEN_HUMID:
      value = AnalogHumidity();
      unit = " %";
      break;

    case SCREEN_LIGHT:
      value = Light();
      unit = " lux";
      break;

    default:
      Serial.println("Unknown screen");
      value = -1;
      unit = " null";
      break;
  }
}

```



```

if (abs(value - old_value) > 0.1) {
    old_value = value;
    //tft.fillScreen(HX8357_BLACK);
    tft.setCursor(40, 60);
    tft.setTextColor(HX8357_WHITE);
    tft.setTextSize(fontSize);

    displayMeasure(value, unit);
}
}

float DigitalTemperature() {
    int firstByte;
    int secondByte;
    float temp = 0;

    delay(1000);

    Wire.beginTransmission(DS1621_ADDR);
    Wire.write(0xAA);
    Wire.endTransmission();
    Wire.requestFrom(DS1621_ADDR, 2);

    firstByte = Wire.read();
    secondByte = Wire.read();

    temp = firstByte;

    if(secondByte)
        temp += 0.5;

    Serial.print("Raw bytes: ");
    Serial.print(firstByte, HEX);
    Serial.print(" ");
    Serial.println(secondByte, HEX);

    Serial.print("Temp: ");
    Serial.println(temp);
    return temp;
}

float AnalogHumidity() {
    int raw = analogRead(PinAnalogHumidity);
    float voltage = raw * 3.3 / 1023; //CONTROLLARE 330
    float humidity = (voltage - 0.8) * 100 / 3.2;
    Serial.print("Humidity: ");
    Serial.println(humidity);
    delay(1000);
    return humidity;
}

float Light() {
    byte firstByte, secondByte;
    int lum;

    Wire.beginTransmission(TSL2561_ADDR);
    Wire.write (TSL2561_REGISTER_ADC0_LSB);
    Wire.endTransmission();
    Wire.requestFrom(TSL2561_ADDR, 1);
    firstByte = Wire.read();

    Wire.beginTransmission(TSL2561_ADDR);
    Wire.write (TSL2561_REGISTER_ADC0_MSB);
    Wire.endTransmission();
    Wire.requestFrom(TSL2561_ADDR, 1);
    secondByte = Wire.read();
    lum = 256*int(secondByte) +int(firstByte);
    Serial.print("Luminosity: ");
    Serial.println(lum);
    return lum;
}

```

```

}

// display functions
void changeDisplay() {
    currentScreen = static_cast<Screen>((currentScreen + 1) % 3);
    old_value = -1000;
    tft.fillRect(0, 0, tft.width(), tft.height(), HX8357_BLACK);

    const char* text;
    switch (currentScreen) {
        case SCREEN_TEMP:
            text = "Temperature:";
            break;
        case SCREEN_HUMID:
            text = "Humidity:";
            break;
        case SCREEN_LIGHT:
            text = "Light Level:";
            break;
        default:
            text = "Unknown screen";
            Serial.println("Unknown screen");
            break;
    }

    displayText(text);
}

void displayText(const char* text) {
    tft.setTextSize(fontSize);
    tft.setTextColor(HX8357_WHITE);

    int16_t x = getX(text);
    int16_t y = 30; // Top margin

    tft.setCursor(x, y);
    tft.print(text);
}

void displayMeasure(float value, const char* unit) {
    tft.setTextSize(fontSize);
    tft.setTextColor(HX8357_GREEN);

    char valueStr[20];
    snprintf(valueStr, sizeof(valueStr), "%.1f", value); // just the number

    char fullStr[30];
    snprintf(fullStr, sizeof(fullStr), "%s%s", valueStr, unit); // number + unit

    int16_t x = getX(fullStr);
    int16_t y = 140;

    tft.fillRect(0, y, tft.width(), 30, HX8357_BLACK);

    tft.setCursor(x, y);
    tft.print(fullStr);

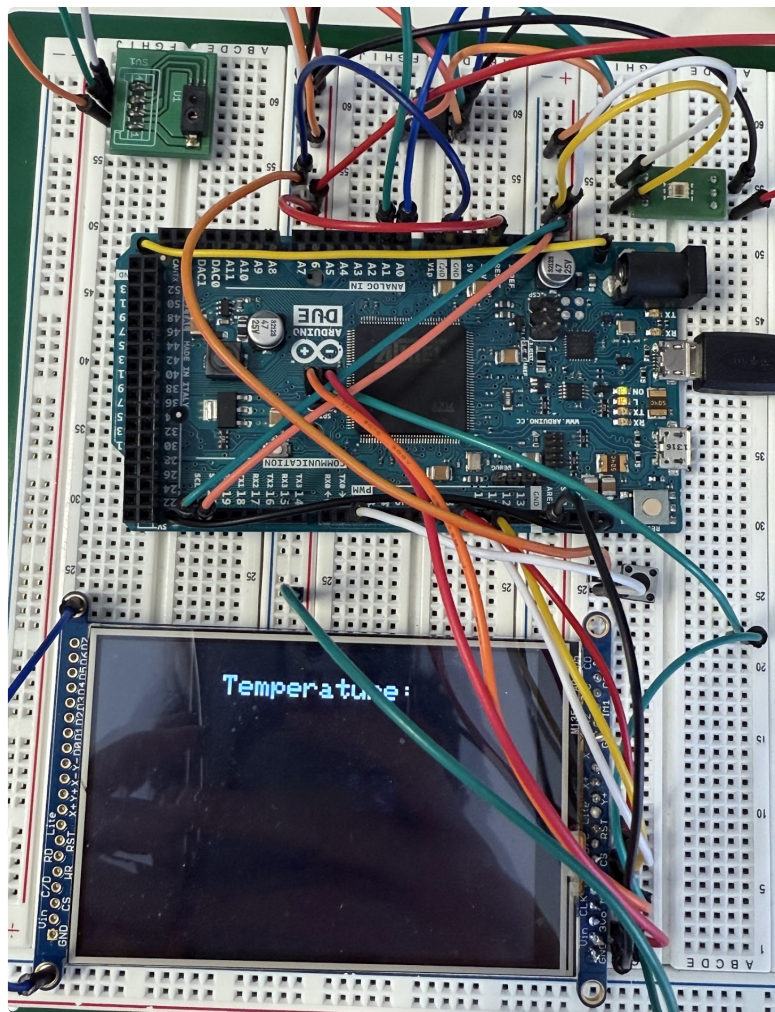
    x = x + 6 + (6*fontSize*strlen(fullStr)); //set x to: old x + 6 (padding) +
    charlength*font*string
    tft.drawBitmap(x, y, epd_bitmap_allArray[currentScreen], 24, 24, HX8357_WHITE);
}

int16_t getX(const char* fullStr){
    int16_t x1, y1;
    uint16_t w, h;
    tft.getTextBounds(fullStr, 0, 0, &x1, &y1, &w, &h);

    return (tft.width() - w) / 2;
}

```

Here are our results:



Circuit

1.2) Second Experiment

The purpose of the experiment is to build the circuit and write a program that reads the ambient temperature by using a TMP36 sensor and graphically displays the value on the TFT display using a colorbar. The position of the cursor indicates the temperature. The image is illustrative: the colorbar can be defined as desired.

The temperature value should be displayed up to the first decimal digit.



Example

```
// DEFINIZIONE COLORI
#define HX8357_BLACK 0x0000
#define HX8357_BLUE 0x001F
```



```

#define HX8357_RED      0xF800
#define HX8357_GREEN    0x07E0
#define HX8357_CYAN     0x07FF
#define HX8357_MAGENTA  0xF81F
#define HX8357_YELLOW   0xFFE0
#define HX8357_WHITE    0xFFFF

// INCLUSIONE LIBRERIE
#include <SPI.h>
#include "Adafruit_GFX.h"
#include "Adafruit_HX8357.h"
#include <Wire.h>

// DEFINIZIONE PIN DISPLAY
#define TFT_CS    10
#define TFT_DC    9
#define TFT_RST   8

Adafruit_HX8357 tft = Adafruit_HX8357(TFT_CS, TFT_DC, TFT_RST);

float reading;
float old_temperature=0;
int PinTemperature = A0;

void setup() {
  tft.begin();
  tft.setRotation(1);
  tft.fillScreen(HX8357_BLACK);
  tft.setCursor(60, 0);
  tft.setTextColor(HX8357_GREEN);
  tft.setTextSize(6);
  tft.println("Ambient");
  tft.setCursor(30, 60);
  tft.setTextColor(HX8357_WHITE);
  tft.println("Temperature");
}

void loop() {
  reading = analogRead(PinTemperature);
  float temperature = (reading - 155) * 330 / 1023.0;

  // Clear only the text area (adjust size and position as needed)
  tft.fillRect(0, 120, 480, 80, HX8357_BLACK);

  old_temperature = temperature;

  String amplitude = String(temperature);
  amplitude = "T = " + amplitude.substring(0, 4) + "C";

  tft.setTextSize(6);
  tft.setTextColor(HX8357_RED);
  tft.setCursor(60, 120); // Set cursor to a known position
  tft.print(amplitude);

  //delay(2000);

  // Draw bar
  int barX = 20;
  int barY = 270;
  int barWidth = 440;
  int barHeight = 20;

  // Clear the whole bar area, including old cursor
  tft.fillRect(0, barY - 10, 440, barHeight + 20, HX8357_BLACK);

  tft.drawRect(barX, barY, barWidth, barHeight, HX8357_WHITE);

```

```

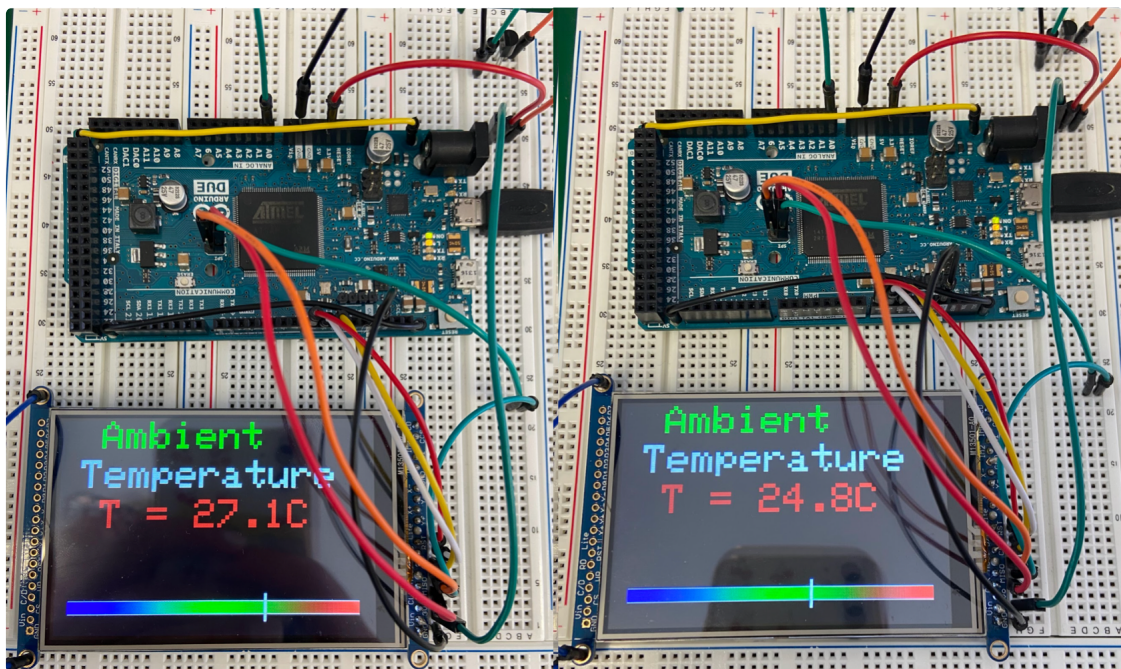
for (int i = 0; i < barWidth; i++) {
    uint8_t r, g, b;
    if (i < barWidth / 2) {
        r = 0;
        g = map(i, 0, barWidth / 2, 0, 255);
        b = map(i, 0, barWidth / 2, 255, 0);
    } else {
        r = map(i, barWidth / 2, barWidth - 1, 0, 255);
        g = map(i, barWidth / 2, barWidth - 1, 255, 0);
        b = 0;
    }
    uint16_t color = tft.color565(r, g, b);
    tft.drawFastVLine(barX + i, barY + 1, barHeight - 2, color);
}

// Draw cursor
float tempMin = 0.0;
float tempMax = 40.0;
float clampedTemp = constrain(temperature, tempMin, tempMax);
int cursorX = map(clampedTemp, tempMin, tempMax, barX, barX + barWidth - 1);

tft.fillRect(cursorX - 2, barY - 10, 4, barHeight + 20, HX8357_WHITE);

delay(1000);
}

```



Display after and before heating the sensor